C Programming: Structures and Unions

A. Topics

1. Structure (struct)

- A structure is like a **box that can hold different types of data together**.
- **Example:**

```c
struct Student {
    char name[50];  // text
    int age;        // number
    float marks;    // decimal number
};
```

- **Access members:** Use `.` (dot)

```c
struct Student s1;
s1.age = 20;
s1.marks = 85.5;
```

2. Nested Structure

- A structure **inside another structure**.

```c
struct Date {
    int day;
    int month;
    int year;
};

struct Student {
    char name[50];
    struct Date dob;  // nested structure
};
```

- **Access nested member:**

```c
s1.dob.day = 23;
s1.dob.month = 1;
s1.dob.year = 2026;
```

3. Union

- A union is like a **structure but only one value can be stored at a time**.

```
union Data {
    int i;
    float f;
    char str[20];
};
```

B. Activity: Array of Books

- Define a **book** using `struct` and store many books in an array.

```
#include <stdio.h>
#include <string.h>

struct Book {
    char title[50];
    char author[50];
    int pages;
    float price;
};

int main() {
    struct Book library[2];  // array of 2 books

    // Book 1
    strcpy(library[0].title, "C Programming");
    strcpy(library[0].author, "Dennis Ritchie");
    library[0].pages = 350;
    library[0].price = 300;

    // Book 2
    strcpy(library[1].title, "Let Us C");
    strcpy(library[1].author, "Yashavant Kanetkar");
    library[1].pages = 400;
    library[1].price = 250;

    // Show books
    for(int i = 0; i < 2; i++) {
        printf("%s by %s, Pages: %d, Price: %.2f\n",
               library[i].title, library[i].author,
               library[i].pages, library[i].price);
    }
```

```
    return 0;
}
```

Explanation:

- `struct Book` = a box for book info.
- `library[2]` = array of 2 boxes.
- Use `strcpy` for text, `.` to access info.
- Loop prints all books.